



Mata Kuliah : Pemrograman Mobile
Program Studi : D4 – Teknik Informatika
Semester : 5

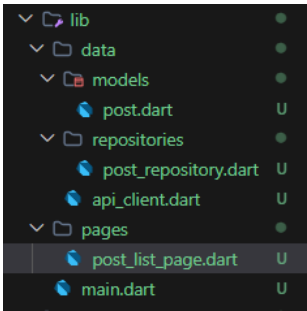
Kelas : 2A TI
NIM : 244107020112
Nama : Muhammad Shabran
Pertemuan : 4

LAPORAN PRAKTIKUM

Aplikasi multi-page dengan GoRouter

Langkah	Keterangan
1.	Persiapan Proyek <pre>PS C:\Users\muham\Documents\Tugas Kuliah\Semester 5\Mobile\244107020112_mobile_course> flutter create week4_api</pre> A new version of Flutter is available! To update to the latest version, run "flutter upgrade". Creating project week4_api... Resolving dependencies in 'week4_api'... (2.5s) Downloading packages... Got dependencies in 'week4_api'. Wrote 131 files. All done! You can find general documentation for Flutter at: https://docs.flutter.dev/ Detailed API documentation is available at: https://api.flutter.dev/ If you prefer video documentation, consider: https://www.youtube.com/c/Flutterdev In order to run your application, type: <pre>\$ cd week4_api \$ flutter run</pre> Your application code is in week4_api\lib\main.dart. <pre>PS C:\Users\muham\Documents\Tugas Kuliah\Semester 5\Mobile\244107020112_mobile_course> PS C:\Users\muham\Documents\Tugas Kuliah\Semester 5\Mobile\244107020112_mobile_course> cd week4_api PS C:\Users\muham\Documents\Tugas Kuliah\Semester 5\Mobile\244107020112_mobile_course\week4_api> flutter pub add dio flutter_riverpod</pre> Resolving dependencies... Downloading packages... + crypto 3.0.7 + cupertino_icons 1.0.9 (2.0.0 available) + dio 5.11.1 + dio_web_adapter 2.2.2 + fixnum 1.1.1 + flutter_riverpod 3.4.3 + http_parser 4.1.2 + listen 1.0.1 + material_color_utilities 0.13.0 (0.13.1 available) + meta 1.18.3 (1.19.0 available) + mime 2.1.0 + riverpod 3.4.3 + state_notifier 1.0.0 + test_api 0.7.12 (0.7.14 available) + typed_data 1.4.0 + uuid 4.6.0 + vector_math 2.4.0 (2.4.3 available) + web 1.1.1 Changed 13 dependencies! 5 packages have newer versions incompatible with dependency constraints. Try 'flutter pub outdated' for more information. <pre>PS C:\Users\muham\Documents\Tugas Kuliah\Semester 5\Mobile\244107020112_mobile_course\week4_api></pre>



	
2.	Model data dengan fromJson aman null <pre>class Post { const Post({ required this.userId, required this.id, required this.title, required this.body, }); final int userId; final int id; final String title; final String body; factory Post.fromJson(Map<String, dynamic> json) { return Post(userId: (json['userId'] as num?)?.toInt() ?? 0, id: (json['id'] as num?)?.toInt() ?? 0, title: json['title'] as String? ?? '', body: json['body'] as String? ?? '',); } Map<String, dynamic> toJson() => { 'userId': userId, 'id': id, 'title': title, 'body': body, }; }</pre>
3.	Konfigurasi Dio terpusat <pre>import 'package:dio/dio.dart'; Dio createDio() { final dio = Dio(BaseOptions(baseUrl: 'https://jsonplaceholder.typicode.com', connectTimeout: const Duration(seconds: 10), receiveTimeout: const Duration(seconds: 10), headers: {'Accept': 'application/json'},),); dio.interceptors.add(LogInterceptor(requestBody: true, responseBody: false),); return dio; }</pre>
4.	Repository sebagai pintu data



```
import 'package:dio/dio.dart';
import '../models/post.dart';

class PostRepository {
  PostRepository(this._dio);
  final Dio _dio;

  Future<List<Post>> fetchPosts() async {
    final response = await _dio.get<List>('/posts');
    final data = response.data ?? [];
    return data
      .whereType<Map<String, dynamic>>()
      .map(Post.fromJson)
      .toList();
  }
}
```

Praktikum 2: Provider dan error handling

Langkah	Keterangan
1.	Provider AsyncNotifier + pesan error ramah pengguna <pre>import 'package:dio/dio.dart'; import 'package:flutter_riverpod/flutter_riverpod.dart'; import 'dart:async'; import 'api_client.dart'; import 'models/post.dart'; import 'repositories/post_repository.dart'; final dioProvider = Provider<Dio>((ref) => createDio()); final postRepositoryProvider = Provider<PostRepository>((ref) => PostRepository(ref.watch(dioProvider))); class PostListNotifier extends AsyncNotifier<List<Post>> { @override Future<List<Post>> build() async { final repository = ref.watch(postRepositoryProvider); return repository.fetchPosts(); } Future<void> refresh() async { state = const AsyncLoading(); try { final repository = ref.read(postRepositoryProvider); state = AsyncData(await repository.fetchPosts()); } catch (e, st) { state = AsyncError(e, st); } } } final postListProvider = AsyncNotifierProvider<PostListNotifier, List<Post>>(() { return PostListNotifier(); }); String friendlyErrorMessage(Object error) { if (error is DioException) { switch (error.type) { case DioExceptionType.connectionTimeout: case DioExceptionType.sendTimeout: case DioExceptionType.receiveTimeout: return 'Koneksi lambat atau timeout. Periksa internet Anda lalu coba lagi.'; case DioExceptionType.connectionError: return 'Tidak dapat terhubung ke server. Periksa internet Anda.'; case DioExceptionType.badResponse: } } }</pre>



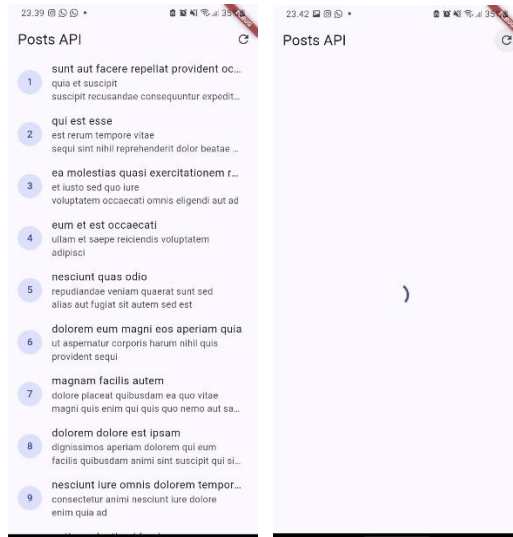
	<pre>case DioExceptionType.badResponse: final code = error.response?.statusCode; if (code == 404) return 'Data tidak ditemukan (404).'; if (code == 401 code == 403) { return 'Akses ditolak (\$code). Periksa kredensial Anda.'; } return 'Server bermasalah (\$code). Coba lagi nanti.'; default: return 'Terjadi kesalahan jaringan. Coba lagi.'; } return 'Terjadi kesalahan tak terduga: \$error';</pre>
2.	UI: loading, error, empty, success <pre>import 'package:flutter/material.dart'; import 'package:flutter_riverpod/flutter_riverpod.dart'; import '../data/providers.dart'; class PostListPage extends ConsumerWidget { const PostListPage({super.key}); @override Widget build(BuildContext context, WidgetRef ref) { final postsAsync = ref.watch(postListProvider); return Scaffold(appBar: AppBar(title: const Text('Posts API'), actions: [IconButton(icon: const Icon(Icons.refresh), onPressed: () => ref.read(postListProvider.notifier).refresh(),),],), body: postsAsync.when(loading: () => const Center(child: CircularProgressIndicator()), error: (err, _) => Center(child: Padding(padding: const EdgeInsets.all(24), child: Column(mainAxisAlignment: MainAxisAlignment.min, children: [Text(friendlyErrorMessage(err), textAlign: TextAlign.center), const SizedBox(height: 12), FilledButton(onPressed: () => ref.invalidate(postListProvider), child: const Text('Coba lagi'),),],),),), data: (posts) { if (posts.isEmpty) { return const Center(child: Text('Belum ada data dari server.')); } return RefreshIndicator(onRefresh: () => ref.read(postListProvider.notifier).refresh(), child: ListView.builder(itemCount: posts.length, itemBuilder: (context, index) { final post = posts[index]; return ListTile(leading: CircleAvatar(child: Text(post.id.toString()),), title: Text(post.title, maxLines: 1, overflow: TextOverflow.ellipsis), subtitle: Text(post.body, maxLines: 2, overflow: TextOverflow.ellipsis),); },),); },),); } }</pre>
3.	Entry point dengan ProviderScope <pre>import 'package:flutter/material.dart'; import 'package:flutter_riverpod/flutter_riverpod.dart'; import 'pages/post_list_page.dart'; void main() => runApp(const ProviderScope(child: MyApp())); class MyApp extends StatelessWidget { const MyApp({super.key}); @override Widget build(BuildContext context) => MaterialApp(title: 'Week 4 - REST API', theme: ThemeData(colorSchemeSeed: Colors.indigo, useMaterial3: true), home: const PostListPage(),); }</pre>



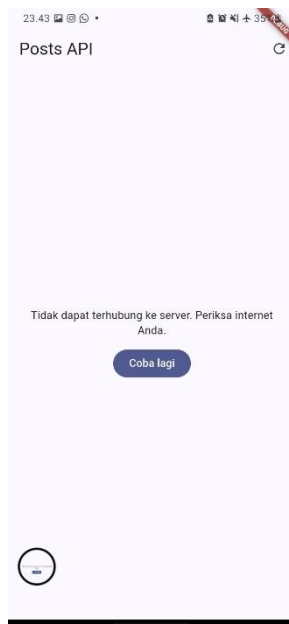
4.

Uji tiga skenario error

1. Jalankan aplikasi dengan internet normal, amati loading lalu daftar 100 posts.



2. Matikan internet (mode pesawat), tekan refresh, amati pesan ramah + tombol Coba lagi. Nyalakan kembali internet, tekan Coba lagi.



3. Sementara ubah baseUrl menjadi URL salah, amati pesan error koneksi. Kembalikan setelah uji.

```
lib/data/api_client.dart:6:16: Error: Undefined name 'baseUrl'.
  baseUrl: baseUrl: '[https://jsonplaceholder.typicode.com_salah](https://jsonplaceholder.typicode.com_salah)',
  ^^^^^^^
lib/data/api_client.dart:6:23: Error: Expected ')' before this.
  baseUrl: baseUrl: '[https://jsonplaceholder.typicode.com_salah](https://jsonplaceholder.typicode.com_salah)',
  ^
Performing hot reload...
Try again after fixing the above error(s).
```



Praktikum 3: Pagination dasar

Langkah	Keterangan
1.	Repository paginated <pre>import 'package:dio/dio.dart'; import '../models/post.dart'; class PostRepository { PostRepository(this._dio); final Dio _dio; Future<List<Post>> fetchPosts() async { final response = await _dio.get<List>('/posts'); final data = response.data ?? []; return data .whereType<Map<String, dynamic>>() .map(Post.fromJson) .toList(); } Future<List<Post>> fetchPostsPage({ required int page, int limit = 10, }) async { final response = await _dio.get<List>('/posts', queryParameters: {'_page': page, '_limit': limit},); final data = response.data ?? []; return data .whereType<Map<String, dynamic>>() .map(Post.fromJson) .toList(); } }</pre>
2.	Notifier dengan state halaman <pre>import 'package:flutter_riverpod/flutter_riverpod.dart'; import 'models/post.dart'; import 'providers.dart'; class PagedPostsState { const PagedPostsState({ this.items = const [], this.page = 0, this.isLoadingMore = false, this.hasMore = true, this.error, }); final List<Post> items; final int page; final bool isLoadingMore; final bool hasMore; final Object? error; }</pre>
3.	Notifier dengan state halaman (lanjutan)



	<pre>class PagedPostsNotifier extends Notifier<PagedPostsState> { @override PagedPostsState build() { Future.microtask(loadFirstPage); return const PagedPostsState(); } Future<void> loadFirstPage() async { final repository = ref.read(postRepositoryProvider); try { final items = await repository.fetchPostsPage(page: 1, limit: 10); state = PagedPostsState(items: items, page: 1, hasMore: items.length == 10,); } catch (e) { state = PagedPostsState(error: e); } } Future<void> loadNextPage() async { if (state.isLoadingMore !state.hasMore) return; final repo = ref.read(postRepositoryProvider); final currentItems = state.items; final currentPage = state.page; state = PagedPostsState(items: currentItems, page: currentPage, isLoadingMore: true, hasMore: state.hasMore,); try { final next = currentPage + 1; final items = await repo.fetchPostsPage(page: next, limit: 10); state = PagedPostsState(items: [...currentItems, ...items], page: next, hasMore: items.length == 10,); } catch (e) { state = PagedPostsState(items: currentItems, page: currentPage, error: e,); } } final pagedPostsProvider = NotifierProvider<PagedPostsNotifier, PagedPostsState>(PagedPostsNotifier.new); }</pre>
4.	UI infinite scroll



	<pre> import 'package:flutter/material.dart'; import 'package:flutter_riverpod/flutter_riverpod.dart'; import '../data/paged_posts.dart'; import '../data/providers.dart'; class PagedPostPage extends ConsumerStatefulWidget { const PagedPostPage({super.key}); @override ConsumerState<PagedPostPage> createState() => _PagedPostPageState(); } class _PagedPostPageState extends ConsumerState<PagedPostPage> { final _controller = ScrollController(); @override void initState() { super.initState(); _controller.addListener(() { if (_controller.position.pixels >= _controller.position.maxScrollExtent - 200) { ref.read(pagedPostsProvider.notifier).loadNextPage(); } }); } @override void dispose() { _controller.dispose(); super.dispose(); } @override Widget build(BuildContext context) { final state = ref.watch(pagedPostsProvider); if (state.error != null && state.items.isEmpty) { return Scaffold(appBar: AppBar(title: const Text('Posts Paged')), body: Center(child: Column(mainAxisAlignment: MainAxisAlignment.min, children: [Text(friendlyErrorMessage(state.error!)), const SizedBox(height: 12),],),),); } return Scaffold(appBar: AppBar(title: const Text('Posts Paged')), body: ListView.builder(controller: _controller, itemCount: state.items.length + 1, itemBuilder: (context, index) { if (index == state.items.length) { if (!state.hasMore) { return const Padding(padding: EdgeInsets.all(16), child: Center(child: Text('Semua data termuat.')),); } return const Padding(padding: EdgeInsets.all(16), child: Center(child: CircularProgressIndicator()),); } final post = state.items[index]; return ListTile(leading: CircleAvatar(child: Text(post.id.toString()),), title: Text(post.title, maxLines: 1, overflow: TextOverflow.ellipsis,),); },),); } } </pre>
5.	Tampilan

AI Challenge

Langkah	Keterangan
---------	------------



1.

perbaiki error tanpa mengganggu program yang lain cukup error nya saja agar bis di build

Worked for 6m >

Error kompilasi telah diperbaiki agar project dapat di-build dengan normal:

Perbaikan yang Dilakukan:

1. `api_client.dart`
 - Memperbaiki sintaks string URL pada `baseUrl` dari `http://jsonplaceholder.typicode.com` menjadi `'https://jsonplaceholder.typicode.com'`.
 - Menghapus karakter acak (`read`) yang tersisa di baris `headers`.
2. `main.dart`
 - Mengganti nama widget dari `Paged_Post_Page()` menjadi `PagedPostPage()`.
 - Menambahkan import `import 'pages/paged_post_page.dart';`.

Silakan jalankan kembali perintah `flutter run` di terminal Anda.

2 files changed +4 -4 > [Review](#)

Testing

Langkah	Keterangan
---------	------------



1.

```
import 'package:flutter_test/flutter_test.dart';
import 'package:dio/dio.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:week4_api/data/models/post.dart';
import 'package:week4_api/data/providers.dart';
import 'package:week4_api/data/repositories/post_repository.dart';
```

```
class FakePostRepository implements PostRepository {
  FakePostRepository({this.items, this.throwError = false});
  final List<Post>? items;
  final bool throwError;
```

```
@override
Future<List<Post>> fetchPosts() async {
  if (throwError) {
    throw DioException(
      requestOptions: RequestOptions(path: '/posts'),
      type: DioExceptionType.connectionError,
    );
  }
  return items ?? const [];
}
```

```
@override
Future<List<Post>> fetchPostsPage(
  {required int page, int limit = 10}) async {
  return fetchPosts();
}
```

```
@override
Future<Post> fetchPost(int id) async {
  final posts = await fetchPosts();
  return posts.firstWhere(
    (p) => p.id == id,
    orElse: () => Post(id: id, userId: 1, title: '', body: ''),
  );
}
```

```
void main() {
  test('fromJson aman terhadap field yang hilang', () {
    final post = Post.fromJson({'id': 7});
    expect(post.id, 7);
    expect(post.title, '');
    expect(post.userId, 0);
  });
```

```
test('friendlyErrorMessage untuk connection error', () {
  final err = DioException(
    requestOptions: RequestOptions(path: '/posts'),
    type: DioExceptionType.connectionError,
  );
  expect(friendlyErrorMessage(err), contains('terhubung'));
});
```

```
class FakePostRepository implements PostRepository {
  Future<Post> fetchPost(int id) async {
    return posts.firstWhere(
      (p) => p.id == id,
      orElse: () => Post(id: id, userId: 1, title: '', body: ''),
    );
  }
}
```

```
void main() {
  test('fromJson aman terhadap field yang hilang', () {
    final post = Post.fromJson({'id': 7});
    expect(post.id, 7);
    expect(post.title, '');
    expect(post.userId, 0);
  });
```

```
test('friendlyErrorMessage untuk connection error', () {
  final err = DioException(
    requestOptions: RequestOptions(path: '/posts'),
    type: DioExceptionType.connectionError,
  );
  expect(friendlyErrorMessage(err), contains('terhubung'));
});
```

```
test('provider sukses dengan repository palsu', () async {
  final container = ProviderContainer(
    overrides: [
      postRepositoryProvider.overrideWithValue(
        FakePostRepository(items: [
          const Post(
            userId: 1, id: 1, title: 'Tes', body: 'Isi'),
        ]),
      ),
    ],
  );
  addTearDown(container.dispose);
```

```
// Gunakan helper: readPostsOnce (lihat providers.dart).
final posts = await readPostsOnce(container);
expect(posts.length, 1);
expect(posts.first.title, 'Tes');
});
```

```
test('provider error dengan repository palsu', () async {
  final container = ProviderContainer(
    overrides: [
      postRepositoryProvider.overrideWithValue(
        FakePostRepository(throwError: true),
      ),
    ],
  );
  addTearDown(container.dispose);
```

```
// Gunakan helper: readPostsErrorOnce (lihat providers.dart).
final err = await readPostsErrorOnce(container);
expect(err, isA<DioException>());
expect(friendlyErrorMessage(err!), contains('terhubung'));
});
```

```
PS C:\Users\muham\Documents\Tugas Kuliah\Semester 5\Mobile\244107020112_mobile_course\week4_api> flutter analyze
Analyzing week4_api...
No issues found! (ran in 19.7s)
PS C:\Users\muham\Documents\Tugas Kuliah\Semester 5\Mobile\244107020112_mobile_course\week4_api> flutter test
00:04 +5: All tests passed!
```

Refleksi

1. Mengapa UI dilarang memanggil Dio langsung? Apa yang rusak jika aturan ini dilanggar?

Answer:

Antarmuka pengguna (UI) dilarang memanggil Dio secara langsung untuk menjaga prinsip pemisahan tanggung jawab (*Separation of Concerns*) dalam arsitektur aplikasi. Jika aturan ini dilanggar, lapisan visual akan sangat bergantung pada implementasi teknis jaringan, sehingga membuat kode sulit diuji (*unit testing*), sulit diperbarui jika ada pergantian pustaka, dan membuat logika penanganan eror tersebar secara tidak terstruktur di dalam komponen antarmuka.

2. Kapan pagination client-side cukup, dan kapan harus mengandalkan pagination server (_page/_limit)?



Answer:

Pagination client-side cukup digunakan apabila total keseluruhan data relatif kecil, statis, dan aman untuk diunduh sekaligus ke memori perangkat tanpa memicu gangguan performa. Sebaliknya, *pagination server-side* yang mengandalkan parameter khusus (`_page` / `_limit`) wajib digunakan saat menangani data berukuran besar agar tidak menyebabkan *memory leak*, menghemat kuota internet pengguna, dan menjaga aplikasi tetap responsif.

3. Bagaimana exception repository berubah menjadi `AsyncError` tanpa `try/catch` di setiap widget? Kapan `try/catch` eksplisit tetap dibutuhkan?

Answer:

Eksepsi dari *repository* dapat otomatis berubah menjadi `AsyncError` karena *provider* dari Riverpod (seperti `AsyncNotifier`) dirancang untuk menangkap *error* yang terlempar (*thrown*) dari operasi *asynchronous* secara internal lalu mendistribusikannya ke UI. Blok `try/catch` eksplisit tetap dibutuhkan di dalam *notifier* ketika Anda mengeksekusi aksi mutasi data secara manual (misalnya *refresh*, *load more*, atau *submit* formulir) agar Anda bisa mengendalikan *state* sebelumnya atau memberikan penanganan eror yang spesifik pada aksi tersebut.

4. Bagian mana dari hasil AI yang Anda perbaiki, dan mengapa?

Answer:

Berdasarkan proses sebelumnya, bagian dari hasil AI yang diperbaiki meliputi koreksi sintaksis dasar seperti penulisan ganda pada variabel `baseUrl`, penambahan tanda kutip yang hilang pada *string* URL, serta penghapusan kata kunci `const` pada *widget* yang tidak bersifat konstan. Perbaikan ini mutlak diperlukan karena kesalahan-kesalahan tersebut memicu kegagalan kompilasi (*build error*) yang membuat aplikasi sama sekali tidak dapat dijalankan di perangkat.